Uni PHP Blog: многопользовательский блог на Laravel с модерацией, RBAC и безопасной файловой системой

Учебный командный проект по PHP и веб-разработке

Laravel • Blade • PHPUnit • Docker / Sail

Функции проекта

Проект включает посты, комментарии, роли, модерацию, загрузку изображений, обработку ошибок и тесты.

КОНТЕНТ

- Посты
- Комментарии
- Изображения

ДОСТУП

- Гость / author / editor / admin
- RBAC + ACL
- Модерация публикаций

НАДЕЖНОСТЬ

- Custom error pages
- Bug reports
- Feature tests

Личный вклад участников

ФЁДОР ПЕНЬКОВ

Инфраструктура и ядро RBAC

- Окружение и запуск
- Breeze и auth
- RBAC и policy-слой

АЛЕКСАНДР СМИРНОВ

Публичный интерфейс

- Гостевой UI
- Комментарии и show page
- Seeder

ЕВГЕНИЙ ЛАРЮШКИН

Административный модуль

- Админка
- Пользователи
- Смена ролей

АНДРЕЙ МУРОМСКИЙ

Модерация, файлы, тесты

- Draft / published
- Файлы и защита изображений
- Feature tests

ПАВЕЛ РАСКОСТОВ

Обработка ошибок и баг-трекинг

- Кастомные ошибки
- Форма отчета об ошибке
- Просмотр отчетов
- UX при сбоях

Технологический стек

BACKEND

PHP + Laravel

Маршруты, Blade, ORM, middleware, policies

ИНТЕРФЕЙС

Blade + Tailwind + Vite

Простая верстка и сборка без тяжелого фронтенд-стека

КАЧЕСТВО

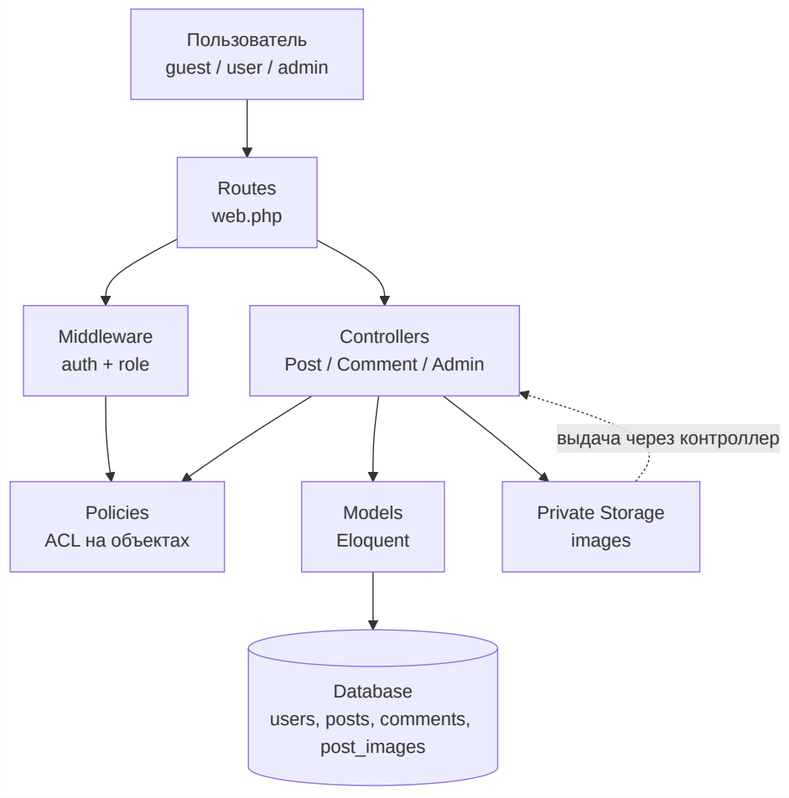
PHPUnit + Feature Tests

Проверка ролей, публикации, файлов и приватного доступа

Laravel Breeze | Docker / Sail | SQLite для тестов | Private file storage

Почему так: использован стандартный стек Laravel без лишних зависимостей и сложной фронтенд-архитектуры.

Запрос и обработка



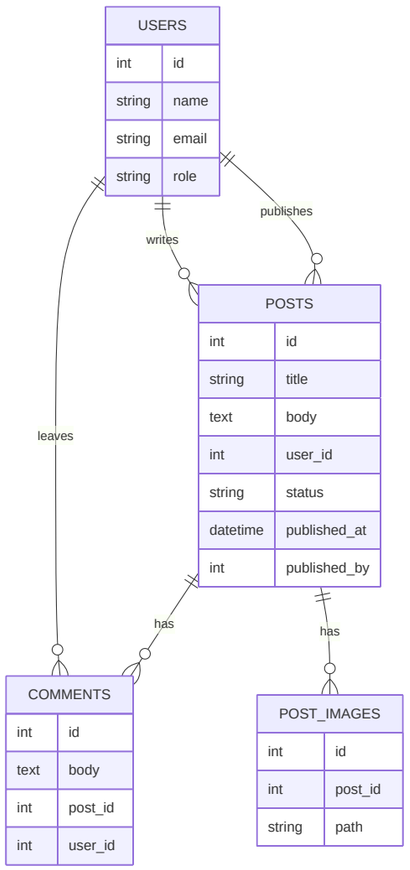
RBAC на уровне разделов

ACL на уровне объекта

Файлы выдаются не напрямую

Таблицы и связи

ER-МОДЕЛЬ



- У поста есть автор и статус
- Комментарии привязаны к посту и пользователю
- Изображения вынесены отдельно
- published_by хранит модератора

> Авторство поста, публикация и файлы разделены на отдельные сущности и поля.

Роли, RBAC и ACL

AUTHOR

- Создает посты
- Редактирует свои draft
- Не публикует

EDITOR

- Смотрит draft
- Publish / unpublish
- Управляет видимостью

ADMIN

- Расширенные права
- Пользователи и роли
- Админский модуль

RBAC

Сначала ограничиваем сам раздел приложения через middleware.

ACL

Потом policy решает, что можно делать с конкретным постом.

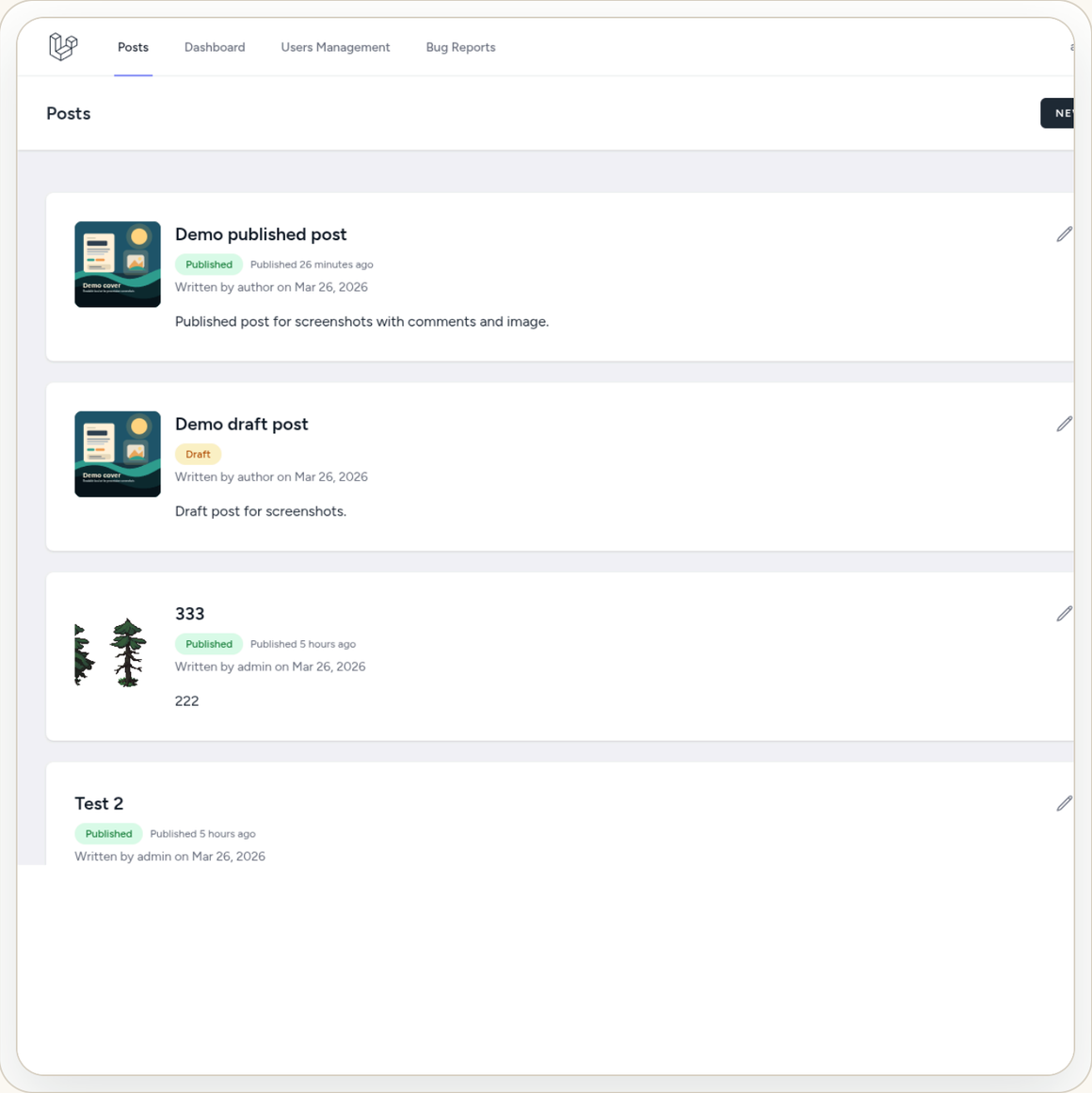
Публичные страницы

ЧТО ДОСТУПНО ГОСТЮ

- Список опубликованных постов
- Просмотр опубликованного поста
- Изображения опубликованных постов
- Вход и регистрация

ЧТО СКРЫТО

- Создание постов
- Черновики и их изображения
- Админские разделы



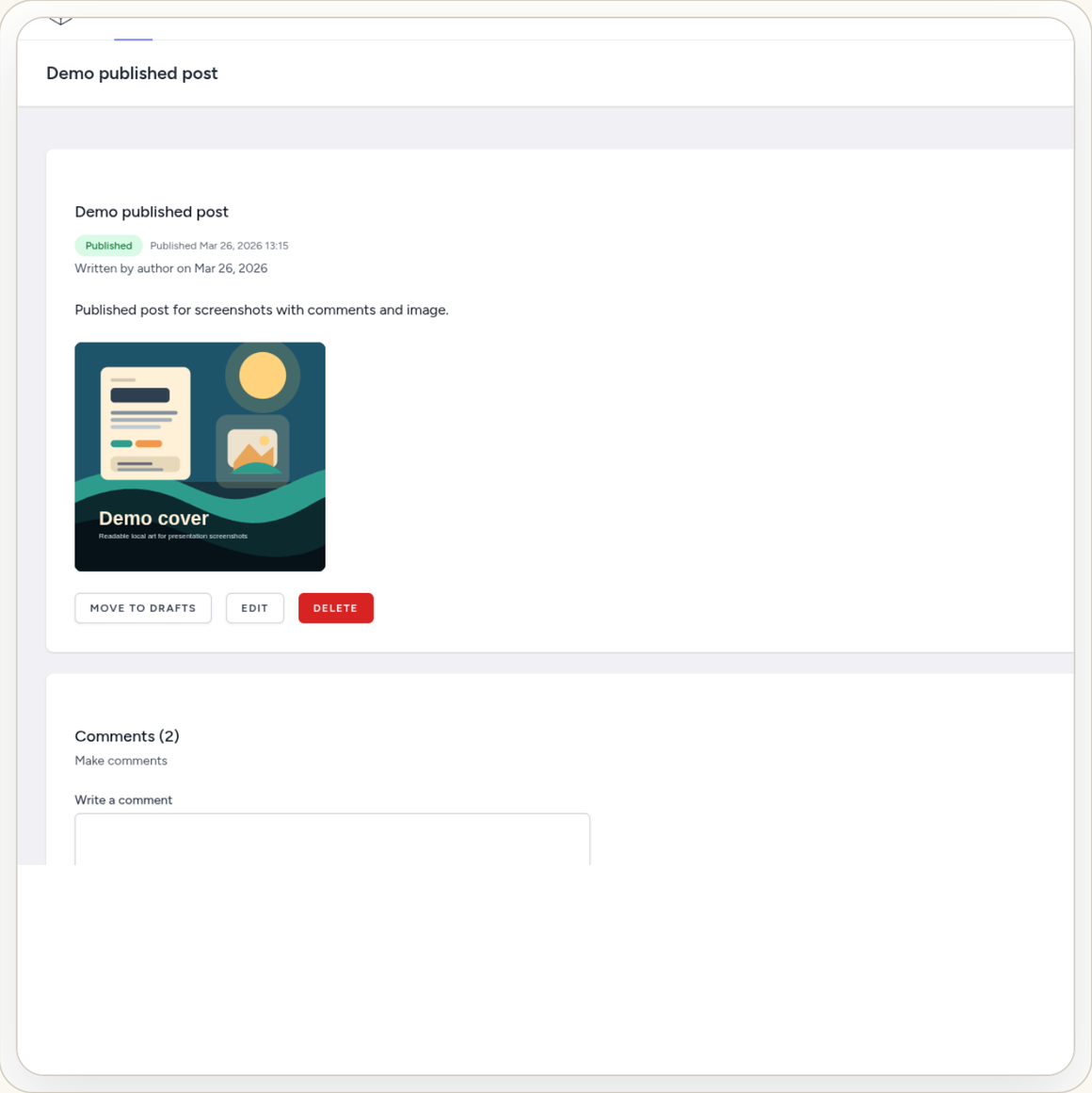
Страница поста и комментарии

КОММЕНТАРИИ

- Связаны с постом и автором
- Добавляются на show page
- Показывают пользовательское взаимодействие

DATABASESEEDER

- Быстрый старт демо
- Наполнение тестовыми данными
- Удобная проверка ролей и UI



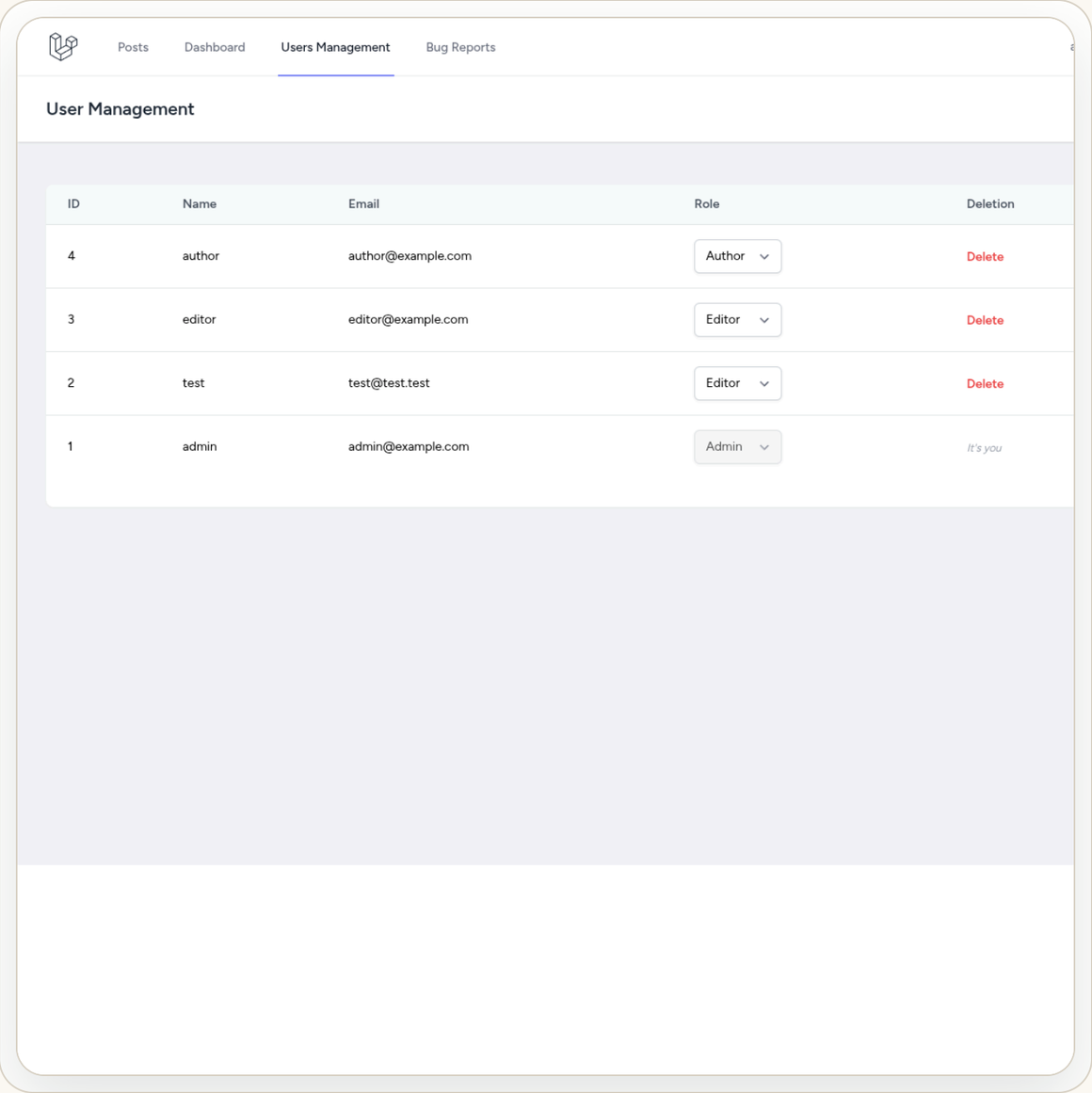
Админка: пользователи и роли

ВОЗМОЖНОСТИ АДМИНИСТРАТОРА

- Список пользователей
- Смена ролей
- Удаление пользователей
- Bug reports

role:user **role:editor** **role:admin**

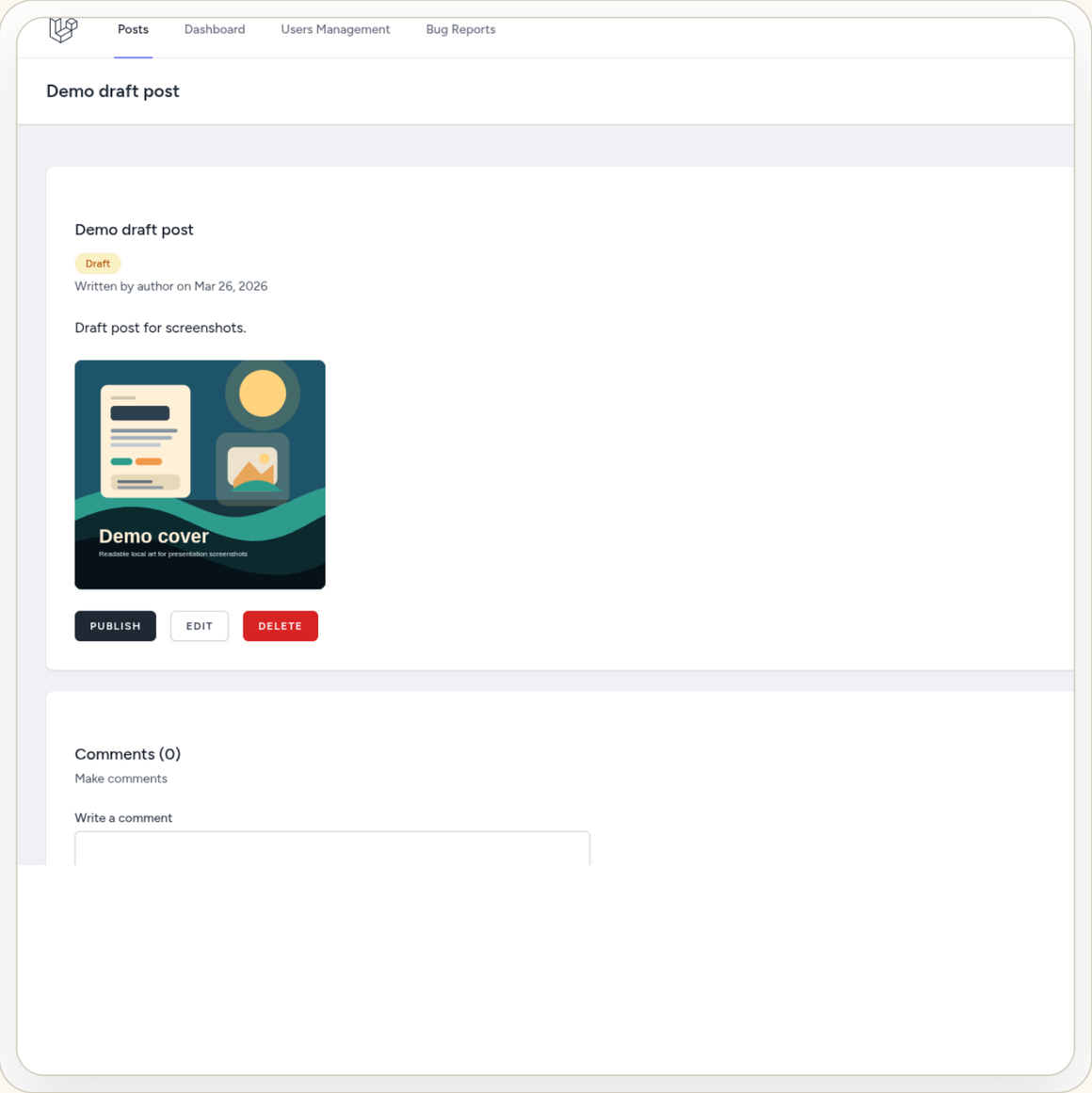
Доступ только для admin.



Черновик и публикация

- Новый пост создается как draft.
- Гость видит только published-посты.
- Publish и unpublish доступны editor и admin.
- Автор редактирует только draft.

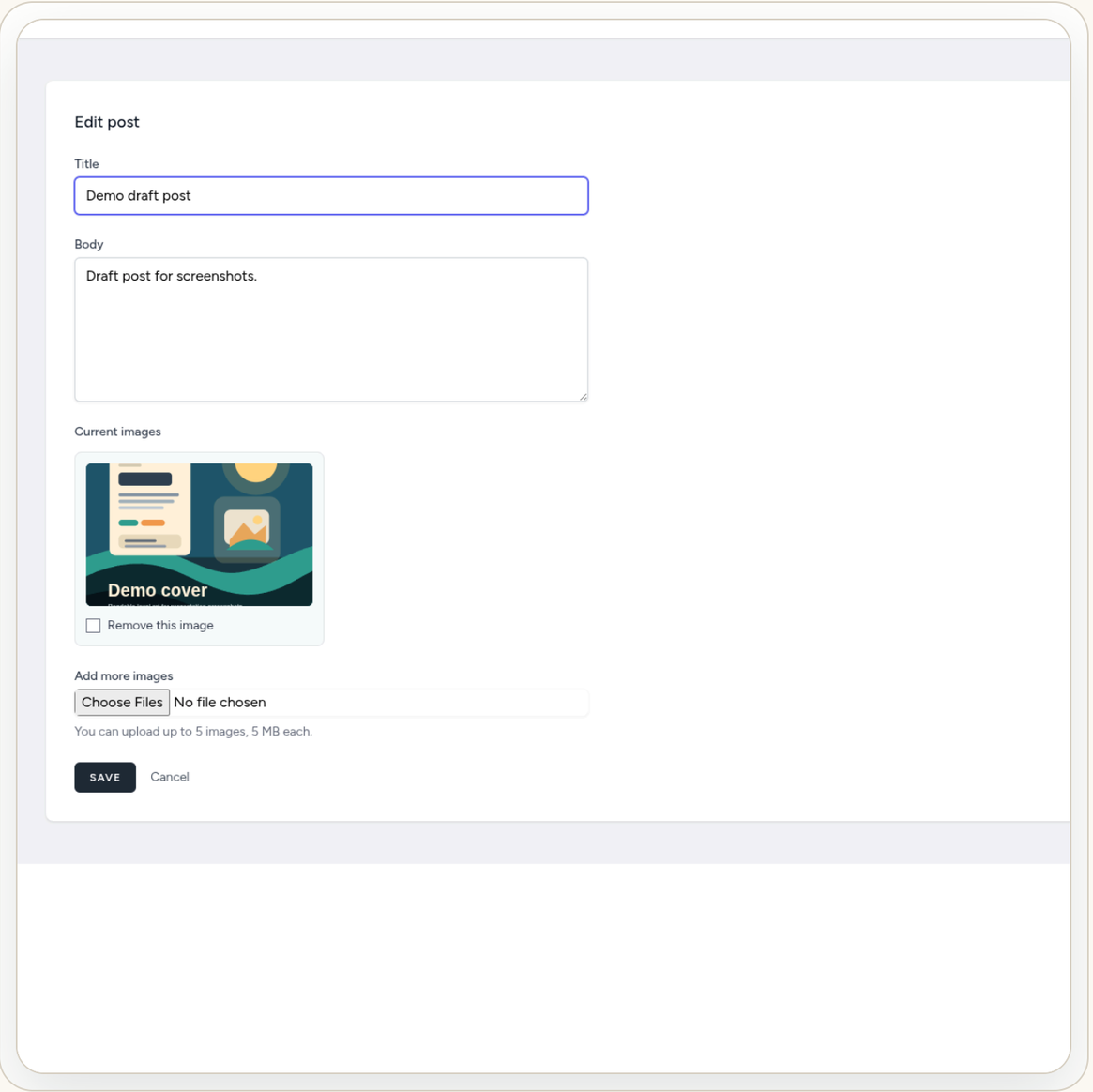
> Справа открыт draft-пост из-под пользователя с правом публикации.



Загрузка и хранение изображений

1. Upload в create / edit post
2. Сохранение в private storage
3. Путь в `post_images`
4. Выдача через `PostImageController` + policy

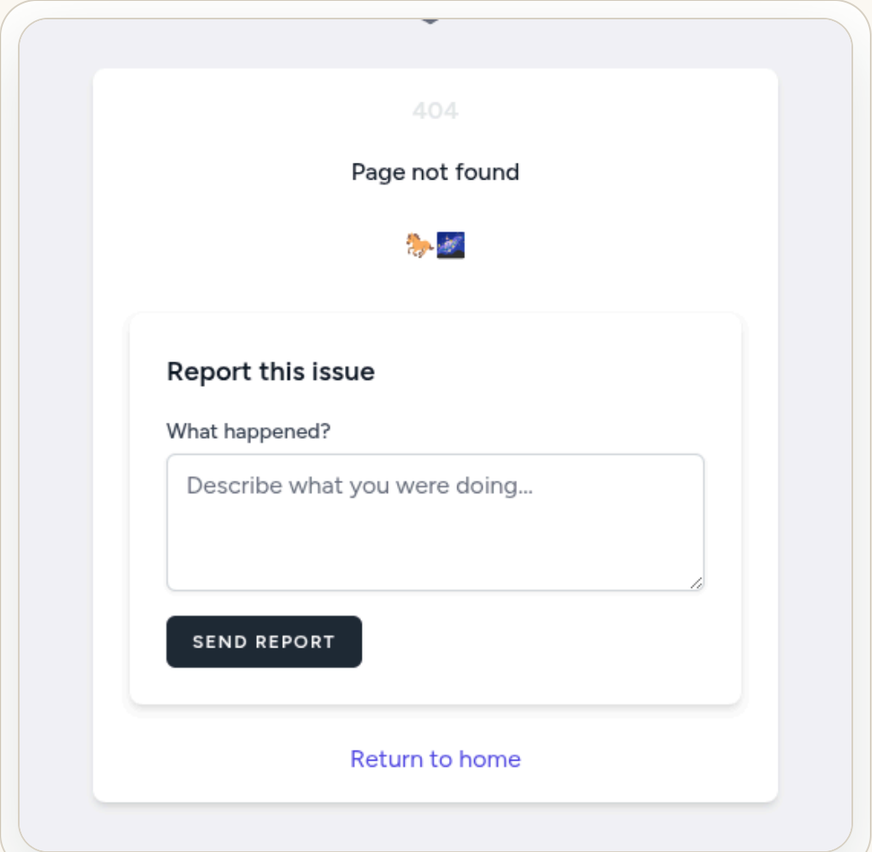
> Файлы не лежат в открытой директории. Доступ к ним идет через контроллер и policy.



Страница 404 и отчет об ошибке

ЧТО РЕАЛИЗОВАНО

- Custom pages: 403 / 404 / 500
- Форма bug report на error page
- Админский просмотр отчетов



Результат: пользователь может отправить отчет, а администратор может его просмотреть.

Что покрывают тесты

- Гость не попадает в create
- Новый пост всегда draft
- Editor делает publish / unpublish
- Автор не меняет published-пост
- Гость не открывает картинки draft
- Изображения можно добавлять и удалять при edit

Feature tests покрывают доступы, модерацию и работу с файлами.

> Критичные места здесь это права доступа и публичность контента.

Что было сделано по структуре

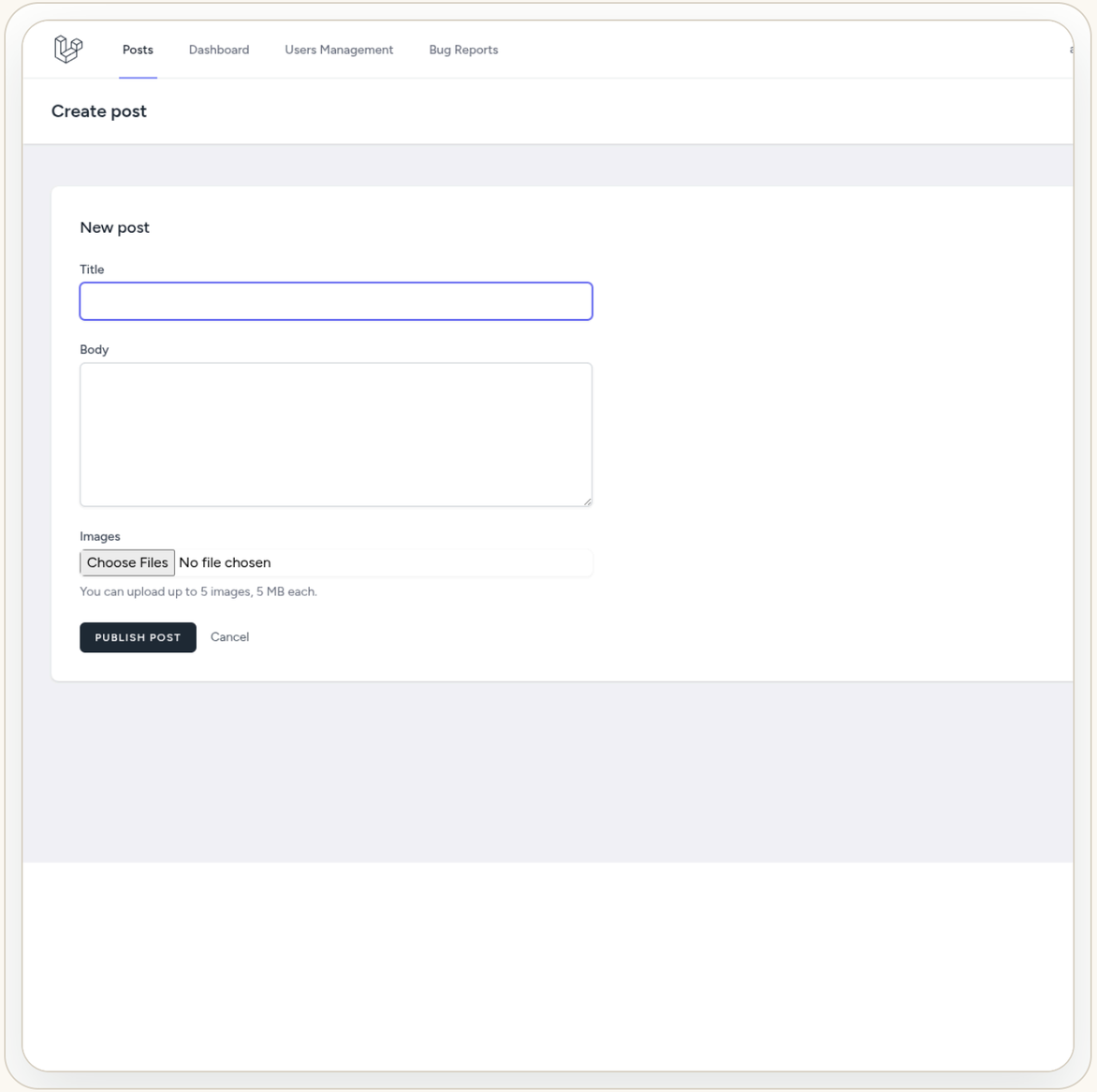
РЕШЕНИЯ

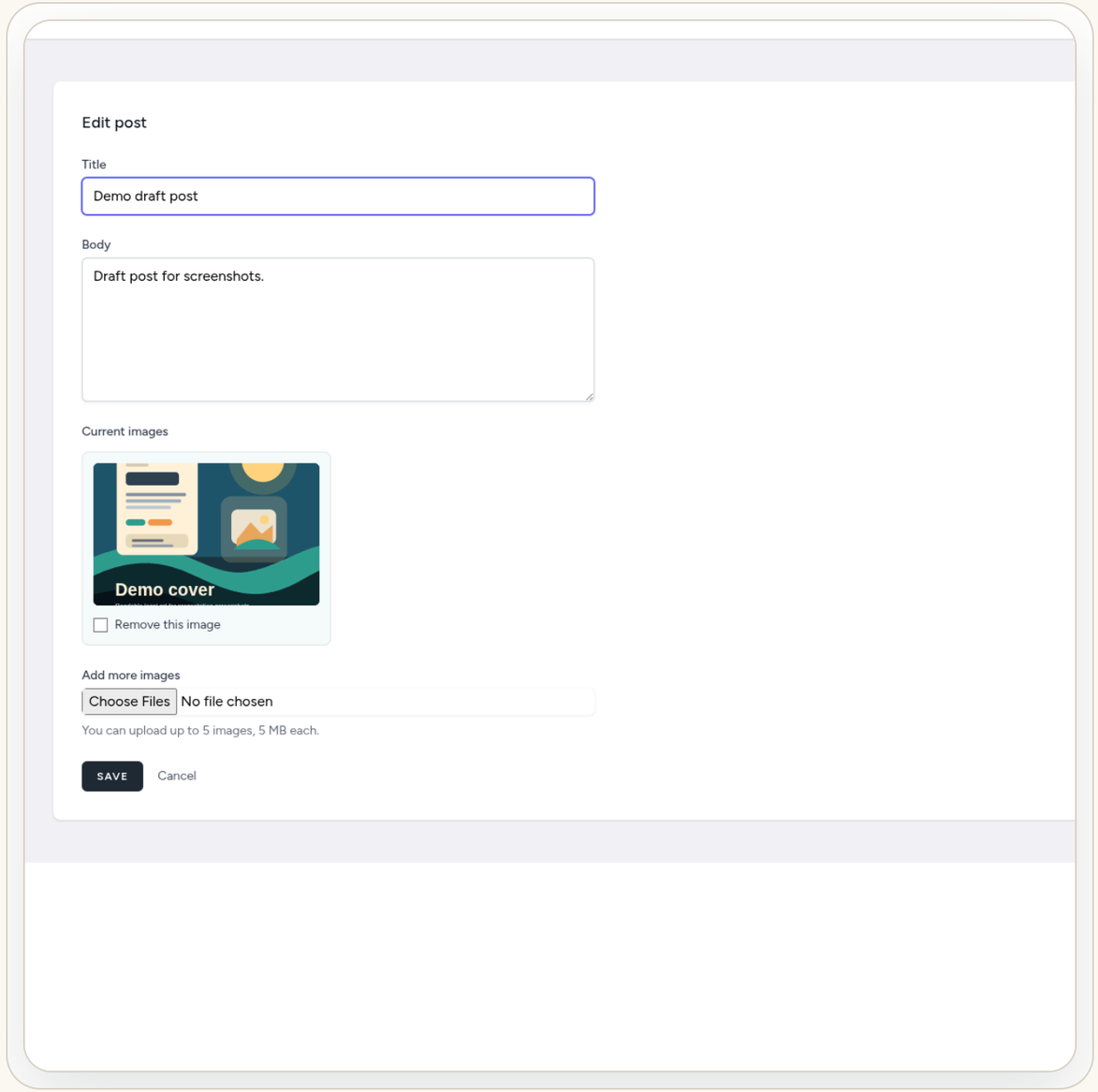
- RBAC отдельно, ACL отдельно
- Статус публикации как бизнес-логика
- Изображения в отдельной таблице
- Файлы через контроллер
- Риски закрыты тестами

ЭФФЕКТ

- Безопасный доступ
- Прозрачная модерация
- Меньше регрессий
- Понятнее логика проекта

Создание и редактирование поста





Что в итоге сделано

ЧТО ВЫПОЛНЕНО

- Командная разработка
- CRUD + auth + роли
- Public UI + admin UI
- Файлы и модерация
- Ошибки и тесты

КЛЮЧЕВОЙ РЕЗУЛЬТАТ

В проекте собраны основные модули, которые требовались для курсовой работы: роли, посты, комментарии, модерация, файлы, ошибки и тесты.

Что важно отметить

Модули проекта связаны между собой через общие маршруты, роли, policies и сценарии использования.

Репозиторий проекта

GITHUB

Основной репозиторий проекта:

https://github.com/Deeplerg/uni-php-blog